

Docker Images

Fullstack feladat

Rostagni Csaba

Az Ön feladata egy dokcer image listaát bemutató weboldal elkészítése. A backend a `http://backend.vm1.test/api` címen érhető el.

Útvonalak

1. Az oldalt egy `container mx-auto` osztályú elembe helyezze el.
2. Valósítsa meg a kliensoldali útvonalválasztást az alábbi táblázat szerint:

Útvonal neve	URL	Cím
<code>images.index</code>	<code>/</code>	Docker Images
<code>images.show</code>	<code>/images/:id</code>	Image

Főoldal

A főoldal feladata, hogy megjelenítse a `DockerImagesTable` komponens segítségével az API-ból betöltött imageket.

3. Az oldal címe legyen *“Docker Images”*, amit formázzon meg az alábbi osztályok használatával:
 - `text-center text-4xl font-bold text-slate-800 my-4`.

Badge komponens

4. Hozza létre a `Badge` komponens, ami átvesz egy `variant` szöveget a szülőtől, alapértelmezett értéke elgen: `primary`.

Hozzon létre egy változót, ami az alábbi objektumot (a lehetséges színvariánsokat) tartalmazza:

```
{
  primary: 'bg-blue-500/10 text-blue-500',
  secondary: 'bg-gray-500/10 text-gray-500',
  success: 'bg-green-500/10 text-green-500',
  danger: 'bg-red-500/10 text-red-500',
  warning: 'bg-yellow-500/10 text-yellow-500',
  info: 'bg-cyan-500/10 text-cyan-500',
  light: 'bg-gray-100 text-gray-700',
  dark: 'bg-gray-800/10 text-gray-800',
}
```

5. Az elem egy `span` elemet tartalmazzon, ami kapja meg a következő osztályokat:
 - `rounded-md px-3 py-1 text-sm font-semibold`
 - a `variant`-tól függően vegye hozzá a megfelelő elemet `variants` objektumból.
6. A tartalmát egy `slot` jelenítse meg.

DockerImagesTable komponens

7. Hozza létre a `DockerImagesTable` komponens, ami átvesz egy `images` tömböt a szülőtől.
8. A táblázatot egy általános tárolóelembe helyezze el. A tárolóelem kapja meg a következő osztályokat:
`w-full overflow-x-auto rounded-lg border border-slate-800 bg-white shadow-sm my-4`.

9. A benne található táblázatot lássa el az alábbi osztályokkal: `min-w-full border-collapse text-sm text-slate-400`
10. A táblázat fejlécét formázza meg az alábbiak szerint:
 - a. A háttér `slate-900` színű, míg a szöveg `slate-200` színű legyen.
 - b. A szövegméret legyen kis, és csupanagybetűs, továbbá balra igazított.
11. Alakítsa ki a fejléccellákat a következő tartalommal: *“ID”, “Repository”, “Tag”, “Méret”, “Létrehozás dátuma”*, majd formázza meg:
 - a. A belső margó legyen `1rem` méretű.
 - b. A betűvastagság legyen `600` értékű.
 - c. Továbbá az utolsó két cella legyen középre igazított.
12. A táblázat adatainak sorait lássa el az alábbi osztályokkal: `group border-b border-slate-800 transition-colors duration-150 hover:bg-slate-200/60`
13. Az első oszlopban jelenjen meg az `img_id` első 22 karaktere, amit 3 pont kövessen.
 - a. Alakítsa linkké, ami az `images.show` oldalra mutat, ahol az `id` paraméter az adott image `id`-ja legyen.
14. Ezt a `repository` kövesse.
15. Készítsen egy függvényt `short` néven, ami az első paraméterül kapott szöveget levágja, hogy legfeljebb annyi karakterből álljon, mint a második paramétere.
 - A hosszba számítson bele a végéhez illesztett `...`
 - Amennyiben rövidebb, mint a megadott szám, úgy teljes egészében adja vissza.
16. A `tag` megjelenítéséhez maximalizálja a hosszt 25 karakterre, ha nincs megadva a tag, akkor a *“default”* szöveg jelenjen meg.
 - Ha az egeret a felhasználó az adatra viszi, akkor jelenjen meg a teljes tag.
17. Készítsen egy függvényt `getVariantBySize` néven, ami a paraméterből kapott méret (MB) alapján meghatározza milyen variánssal kell ajd megjelenítenni az elemet.
 - 200 MB alatt legyen `success`,
 - 500 MB alatt `warning`,
 - ezek felett `danger`
18. Az image méretét jelenítse meg egy `Badge` komponensben a mértékegységgel együtt.
 - a. Amennyiben nincs beállítva az értéke, úgy - jelet jelenítsen meg.
 - b. Határozza meg a megfelelő változatot a `getVariantBySize` függvény felhasználásával.
19. Vegye fel az alábbi függvényt:

```
const formatDate = (dateStr) => {
  if (!dateStr) return '-';
  const date = new Date(dateStr);
  if (isNaN(date.getTime())) return '-';

  return new Intl.DateTimeFormat('hu-HU', {
    year: 'numeric',
    month: '2-digit',
    day: '2-digit',
    hour: '2-digit',
    minute: '2-digit'
  }).format(date);
}
```

20. Jelenítse meg a dátumot az imént létrehozott függvény alkalmazásával.
21. Formázza meg a táblázat adatcelláit:
 - a. Mind kapjon `1rem` kitöltést és legyen `slate-500` színű a szövege.
 - b. Az ID oszlopot kapja mega `font-mono` osztályt.
 - c. Az utolsó két oszlop legyen középre igazítva.

22. Hozzon létre egy reaktív állapotot `images` néven, am az adatok tömbjét fogja kezelni.
23. A Főoldal betöltődésekor kérje le az adatokat az `/api/images` végponttól és tárolja el az `images` állapotban.
24. Jelenítse meg a lekért adatokat a `DockerImagesTable` használatával.

Statisztika

A főoldal tetején a látható képekről kell elkészíteni enémi statisztikát.

25. Hozza létre a `totalNumberOfImages` és `totalImageSize` számított értékeket, ami az `images` állapot alapján határozza meg a ez image-ek számát és összesített méretét.

A Card komponens

26. Hozza létre a `Card` komponens, ami átvesz egy `title` szöveget a szülőtől.
27. A kártya egy `div` ben legyen, melyre az alábbi osztályok érvényesüljenek:
 - `rounded-md border-2 border-slate-800 bg-white p-4`
28. A címet egy bekezdésben jelenítse meg, amire alkalmazza. akövetkező osztályokat:
 - `text-lg font-bold uppercase tracking-wider text-slate-600"`
29. A kártya tartalmát egy `div` elembe helyezze el egy egyszerű `slot` alkalmazásával, amit lásson el az alábbi osztályokkal:
 - `mt-3 text-2xl font-bold text-slate-500`
30. Alakítson ki egy rácsrendszer, ami
 - a. közepes mérettől kezdve 2 oszlopban jelenítse meg az elemeit,
 - b. a rendelkezésre álló hely 50% töltse ki,
 - c. legyen vízszintesen középre igazított,
 - d. az elemek közti távolság legyen `1erm`
31. A rácsrendszerben vegyen fel egy `Card`-ot `"Images"` címmel, a tartalom legyen az elemek száma.
32. Továbbá egy másik `Card`-ot, aminrk a címe `"Size"`, az értéke az összes image összesített mérete legyen.

Egy image oldal

A show oldal feladata, hogy megjelenítse egy image adatait.

33. Az oldal címe legyen `"repository:tag"`, amit formázzon meg az alábbi osztályok használatával:
 - `text-center text-4xl font-bold text-slate-800 my-4.`
34. Hozza létre a `show` oldalt, ami egy `image` nevű állapotban tárolja a megjelenítendő image objektumát.
35. Az oldal betöltődésekor kérje le az adatokat az `/api/images/:id` végponttól és tárolja el az `image` állapotban.
 - Az útvonal paraméterei alapján állapítsa meg az ID-t.
36. Alakítson ki egy rácsrendszer, ami
 - a. közepes mérettől kezdve 3 oszlopban jelenítse meg az elemeit,
 - b. az elemek közti távolság legyen `1erm`
37. Jelenítsen meg 4 kártyát:
 - a. A Repository nevét
 - b. A Tag-et
 - c. Az image méretét MB-ban, mértékegységgel együtt.
 - d. A image létrehozásának a dátumát, idő nélkül.
38. Hozzon létre egy jobbra igazított linket, ami az `images.index` oldalra mutat.
39. Formázza meg a linket:
 - `text-blue-500 hover:text-blue-700 hover:underline mt-4`

Szűrés, rendezés

Bővítse ki a főoldalt, hogy legyen alkalmas szűrésre és rendezésre is.

A Button komponenes

40. Hozza létre a **Card** komponenst, ami átvesz két szöveget **variant** és **type**, illetve egy logikai **disabled** a szülőtől.
 - A **variant** alapértelmezett értéke legyen **primary**
 - A **type** alapértelmezett értéke legyen **submit**
 - A **disabled** alapértelmezett értéke legyen **false**
41. Vegye fel a az alap osztályokat és a lehetséges variánsokat:

```
const baseClasses =
  'inline-flex items-center justify-center gap-2 ' +
  'rounded-md border px-4 py-2 text-sm font-medium ' +
  'transition-colors duration-150 cursor-pointer ' +
  'focus-visible:outline-none focus-visible:ring-2 ' +
  'focus-visible:ring-offset-2 ' +
  'disabled:cursor-not-allowed disabled:opacity-50'

const variants = {
  primary:
    'border-transparent bg-blue-500/10 text-blue-500 hover:bg-blue-500/20 focus-visible:ring-blue-500',
  secondary:
    'border-transparent bg-gray-500/10 text-gray-500 hover:bg-gray-500/20 focus-visible:ring-gray-500',
  success:
    'border-transparent bg-green-500/10 text-green-500 hover:bg-green-500/20 focus-visible:ring-green-500',
  danger:
    'border-transparent bg-red-500/10 text-red-500 hover:bg-red-500/20 focus-visible:ring-red-500',
  warning:
    'border-transparent bg-yellow-500/10 text-yellow-500 hover:bg-yellow-500/20 focus-visible:ring-yellow-500',
  info:
    'border-transparent bg-cyan-500/10 text-cyan-500 hover:bg-cyan-500/20 focus-visible:ring-cyan-500',
  light:
    'border-transparent bg-gray-100 text-gray-700 hover:bg-gray-200 focus-visible:ring-gray-400',
  dark:
    'border-transparent bg-gray-800/10 text-gray-800 hover:bg-gray-800/20 focus-visible:ring-gray-800',

  'outline-primary':
    'border-blue-500 bg-transparent text-blue-500 hover:bg-blue-500/10 focus-visible:ring-blue-500',
  'outline-secondary':
    'border-gray-500 bg-transparent text-gray-500 hover:bg-gray-500/10 focus-visible:ring-gray-500',
  'outline-success':
    'border-green-500 bg-transparent text-green-500 hover:bg-green-500/10 focus-visible:ring-green-500',
  'outline-danger':
    'border-red-500 bg-transparent text-red-500 hover:bg-red-500/10 focus-visible:ring-red-500',
  'outline-warning':
    'border-yellow-500 bg-transparent text-yellow-500 hover:bg-yellow-500/10 focus-visible:ring-yellow-500',
  'outline-info':
    'border-cyan-500 bg-transparent text-cyan-500 hover:bg-cyan-500/10 focus-visible:ring-cyan-500',
  'outline-light':
    'border-gray-300 bg-transparent text-gray-700 hover:bg-gray-100 focus-visible:ring-gray-400',
  'outline-dark':
    'border-gray-800 bg-transparent text-gray-800 hover:bg-gray-800/10 focus-visible:ring-gray-800',
}
```

42. A komponens gyökéreleme egy **button** legyen, ami kapja meg a típust, illetve az alap osztályokon túl a kiválasztott variánsnak megfelelő további osztályokat is. Továbbá a **disabled** értéke tükrözze a **disabled** állapotot.

AcrionBar komponens

43. Hozza létre a **AcrionBar** komponenst, ami képes két irányú kötésre.
44. Hozzon létre egy általános tároló elemet, amit flexbox segítségével formázzon:
 - a. Az elemek közötti távolság legyen **1rem**
 - b. A távolság fennmaradó részét az elemek között ossza fel.
 - c. Amennyiben nem férne ki a tartalom, úgy tördelje.
45. Ezen belül hozzon létre egy flexibilis elemet **1rem**-es távolsággal, amiben a az alábbi gombok szerepeljenek:
 - Név szerint növekvő
 - Név szerint csökkenő
 - Méret szerint növekvő
 - Méret szerint csökkenő
46. Az egyed gombokra kattintva jelezze a komponens a szülő felé az **orderBy**-on keresztül, hogy mi alapján és milyen irányban.
47. A gombok csoportja után helyezzen el egy űrlapot, benne egy input elemet:
 - a. Az alábbi osztályokkal formázva: **border border-slate-500 rounded px-2**
 - b. Kössze össze kétirányú közéssel az inputot és a szülő elemet.
48. Hozzon létre egy számított állapotot **filtered** néven, ami a keresőmező módosítására a feltételeknek megfelelően szűr.
49. Hozzon létre egy számított állapotot **filteredNumberOfImages** néven, a szűrt elemek számát adja meg.
50. Hozzon létre egy számított állapotot **filteredImageSize** néven, a szűrt elemek összesített méretét adja meg.
51. Módosítsa úgy a statisztika kártyákat, hogy a a szűrt érték is megjelenjen rajta.
52. Módosítsa úgy a **DockerImagesTable** komponens paraméterezését, hogy a szűrt adatokat jelenítse meg.
53. Kezelje le az **order-by** eseményét az **ActionBar** komponensnek a **handleOrderBy** függvény által:
 - a. A kapott értékek alapján hívja meg az **/api/images** végpontot, ahol az **orderBy** és az **order** query paraméterként kerüljön átadásnak.
 - b. Cserélje le az **images** állapot jelenlegi értékét az új, rendezett adatokra.